


# HTML & CSS LAB (BCS-155)

## B.Tech CSE 1st SEMESTER

**SUBMITTED TO:**

**MR. AYUSH KUMAR**

**SUBMITTED BY:**

**Abhijeet Kumar**
**Section G**
**Roll No. BCS2026 512**

| SN. | PROGRAM NAME | PAGE | DATE | SIGNATURE |
|---|---|---|---|---|
| 1 | Starting a Git Repository | 3 | | |
| 2 | Pushing and Cloning a Repository | 6 | | |
| 3 | Your First HTML Page | 8 | | |
| 4 | Divs, Attributes and a Small Fix | 10 | | |
| 5 | Links and Images | 12 | | |
| 6 | Types of HTML Lists | 14 | | |
| 7 | About Me Page | 17 | | |
| 8 | Sign-Up Forms and Choices | 20 | | |
| 9 | Tables with Headers and Footers | 23 | | |
| 10 | CSS Fundamentals | 26 | | |
| 11 | Units and CSS Variables | 28 | | |
| 12 | Box Model, Cards and Scrolling | 31 | | |
| 13 | Flexbox Layouts | 34 | | |
| 14 | CSS Grid Layouts | 37 | | |
| 15 | Responsive Pages with Media Queries | 41 | | |

# EXPERIMENT NO. 1

## 1. Experiment Title:

Starting a Git Repository

## 2. Aim:

To start a Git repository for the lab folder and save work with commits.

## 3. Software / Tools Required:

Git (version control), Git Bash / terminal, a text editor.

## 4. Theory:

Git is a version control system that records the history of a project. **git init** turns a folder into a repository. Changes are first placed in the *staging area* with **git add** and are then saved as a snapshot with **git commit -m**. **git config --local** stores the name and email only for this repository, **git status** shows the state of the files and **git log --oneline** lists the commits in short form, newest first.

## 5. Algorithm / Procedure:

1. Create the folder html-lab and go inside it.
2. Run git init, then set user.name and user.email with git config --local and display them.
3. Create index.html with one h1 heading.
4. Run git status, stage the file with git add and commit it with the message "Add home page".
5. Add an h2 heading to index.html, stage it and commit it with the message "Add next meeting heading".
6. Run git log --oneline to see both commits.

## 6. Program / Queries / Commands:

**Task 1:** Create the repository and set the local name and email.

```
mkdir html-lab
cd html-lab
git init
git config --local user.name "Abhijeet Kumar"
git config --local user.email "abhijeet@example.com"
git config user.name
git config user.email
```

**Task 2:** Create index.html, check the status and make the first commit.

```
echo "<h1>Welcome to HTML Lab</h1>" > index.html
git status
git add index.html
git commit -m "Add home page"
```

**Task 3:** Add an h2 heading, commit it and list the history.

```
echo "<h2>Next meeting: Friday</h2>" >> index.html
git add index.html
git commit -m "Add next meeting heading"
git log --oneline
```

## 7. Input:

Commit messages: Add home page, Add next meeting heading.

## 8. Output:

**Task 1:** Each git config command prints the value that was set.

```
f MINGW64:/c/Users/abhijeet/html-lab

abhijeet@LAPTOP MINGW64 ~
$ mkdir html-lab

abhijeet@LAPTOP MINGW64 ~
$ cd html-lab

abhijeet@LAPTOP MINGW64 ~/htmlL- lab
$ git init
Initialized empty Git repository in C:/Users/abhijeet/html-lab/.git/

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git config --local user.name "Abhijeet Kumar"

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git config --local user.email "abhijeet@example.com"

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git config user.name
Abhijeet Kumar

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git config user.email
abhijeet@example.com
```

**Task 2:** git status lists index.html as untracked. The commit line shows 1 file changed.

```
f MINGW64:/c/Users/abhijeet/html-lab

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ echo "<h1>Welcome to HTML Lab</h1>" > index.html

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git add index.html

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git commit -m "Add home page"
[main (root-commit) 059b0a3] Add home page
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
```

**Task 3:** Two commits are listed, the newest one first.

```
f MINGW64:/c/Users/abhijeet/html-lab

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ echo "<h2>Next meeting: Friday</h2>" >> index.html

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git add index.html

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git commit -m "Add next meeting heading"
[main b9c8c89] Add next meeting heading
 1 file changed, 1 insertion(+)

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git log --oneline
b9c8c89 (HEAD -> main) Add next meeting heading
059b0a3 Add home page
```

## 9. Result:

The Git repository was created in html-lab, the local name and email were set, and two commits were saved. git log --oneline shows both commits, so the experiment was performed successfully.

# EXPERIMENT NO. 2

## 1. Experiment Title:

Pushing and Cloning a Repository

## 2. Aim:

To send commits to a second repository on the same computer and to make a copy of it with clone.

## 3. Software / Tools Required:

Git, Git Bash / terminal.

## 4. Theory:

A *remote* is another copy of a repository. A *bare* repository (git init --bare) has no working files and is used only to share commits. **git remote add origin** gives the remote a short name, **git push** sends local commits to it and **git clone** creates a full copy of a repository, including its history.

## 5. Algorithm / Procedure:

1. Inside html-lab run git init --bare ../html-remote to create an empty shared repository.
2. Add it as origin with git remote add origin ../html-remote and check it with git remote -v.
3. Push the commits with git push origin main (the branch is main, so no change is needed).
4. Go one folder up with cd .. and run git clone html-remote html-copy.
5. Go into html-copy and run git log --oneline to confirm that the same commits are present.

## 6. Program / Queries / Commands:

**Task 1:** Create the bare repository, add it as origin and verify it.

```
git init --bare ../html-remote
git remote add origin ../html-remote
git remote -v
```

**Task 2:** Push the commits to origin.

```
git status
git push origin main
```

**Task 3:** Clone html-remote into html-copy and view the log.

```
cd ..
git clone html-remote html-copy
cd html-copy
git log --oneline
```

## 7. Input:

No input is needed; the repository from Assignment 1 is used.

## 8. Output:

**Task 1:** git remote -v lists origin for fetch and for push.

```
f MINGW64:/c/Users/abhijeet/html-lab

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git init --bare ../html-remote
Initialized empty Git repository in C:/Users/abhijeet/html-remote/

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git remote add origin ../html-remote

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git remote -v
origin  ../html-remote (fetch)
origin  ../html-remote (push)
```

**Task 2:** The push ends with a line that shows the branch: main -> main.

```
f MINGW64:/c/Users/abhijeet/html-lab

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git status
On branch main
nothing to commit, working tree clean

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 480 bytes | 480.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
To ../html-remote
 * [new branch]      main -> main
```

**Task 3:** The same commits appear in html-copy as in html-lab.

```
f MINGW64:/c/Users/abhijeet/html-copy

abhijeet@LAPTOP MINGW64 ~/html-lab (main)
$ cd ..

abhijeet@LAPTOP MINGW64 ~
$ git clone html-remote html-copy
Cloning into 'html-copy'...
done.

abhijeet@LAPTOP MINGW64 ~
$ cd html-copy

abhijeet@LAPTOP MINGW64 ~/html-copy (main)
$ git log --oneline
b9c8c89 (HEAD -> main, origin/main, origin/HEAD) Add next meeting heading
059b0a3 Add home page
```

## 9. Result:

The commits were pushed from html-lab to html-remote and html-remote was cloned into html-copy. Both repositories show the same history, so the experiment was performed successfully.

# EXPERIMENT NO. 3

## 1. Experiment Title:

Your First HTML Page

## 2. Aim:

To write a first HTML page with a title, headings and a paragraph, and to format some text.

## 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

## 4. Theory:

Every HTML page starts with **<!DOCTYPE html>** and has an **html** element that holds a **head** (the title shown on the browser tab) and a **body** (the visible content). Headings **h1** to **h6** go from the largest to the smallest, **p** makes a paragraph, **strong** shows important (bold) text, **em** shows emphasised (italic) text and **br** breaks a line.

## 5. Algorithm / Procedure:

1. Open the code editor and create first.html with the document structure, the title and an h1.
2. Create headings.html with an h1, an h2, an h3 and a paragraph.
3. Create text-format.html with two paragraphs; use strong and em on one word each and a br inside one paragraph.
4. Open each file in the browser and take a screenshot.

## 6. Program / Queries / Commands:

**Task 1:** first.html - document structure, title and one h1.

**first.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 3 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, I am Asha</h1>
</body>
</html>
```

**Task 2:** headings.html - three heading levels and a paragraph.

**headings.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 3 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Headings</title>
</head>
<body>
  <h1>Learning HTML</h1>
  <h2>Why HTML is important</h2>
  <h3>The basic structure</h3>
  <p>HTML is the standard language used to create the structure of web pages.</p>
</body>
</html>
```

**Task 3:** text-format.html - strong, em and a line break.

**text-format.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 3 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Text Formatting</title>
</head>
<body>
  <p>HTML is a <strong>markup</strong> language.<br>
  This sentence starts on a new line.</p>
  <p>Practice makes a programmer <em>perfect</em>.</p>
</body>
</html>
```

## 7. Input:

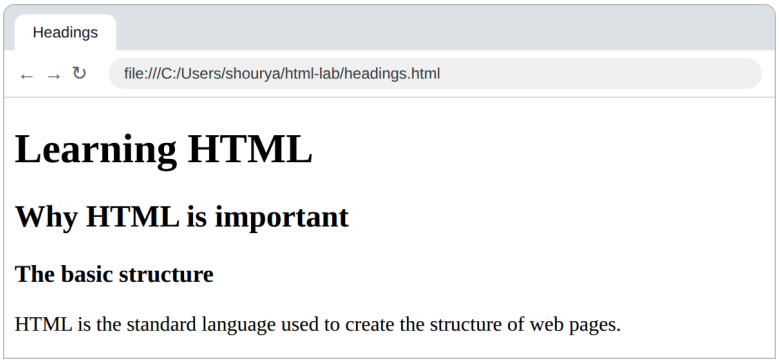
No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

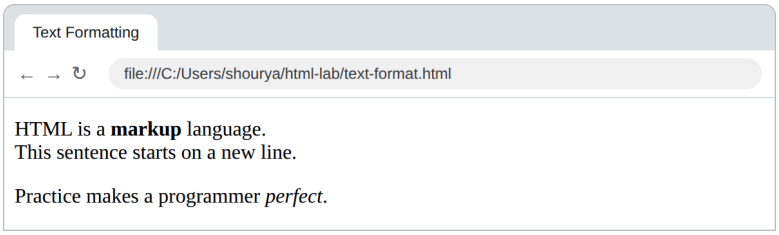
**Task 1:** The browser tab says My First Page. The page shows one large heading.




**Task 2:** Three headings, each smaller than the one above, and a paragraph under them.




**Task 3:** The strong and em words look bold and italic. The line break starts a new line inside the paragraph.




## 9. Result:

Three HTML pages were created and opened in the browser. The title, headings, paragraph, bold text, italic text and line break appeared as expected, so the experiment was performed successfully.

# EXPERIMENT NO. 4

## 1. Experiment Title:

Divs, Attributes and a Small Fix

## 2. Aim:

To group parts of a page with div elements, add a data attribute, and fix the errors in a broken page.

## 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

## 4. Theory:

A **div** is a block container used to group content. The **class** attribute can be shared by many elements, while an **id** must be unique on the page. A **data-*** attribute stores extra information on an element; it is not shown on the page. Tags must be opened and closed correctly (well-formed markup), otherwise the browser has to guess the structure.

## 5. Algorithm / Procedure:

1. Create layout.html with three divs using the classes header, main and footer.
2. Create attributes.html with a div with id profile and a paragraph with data-topic="hobbies" inside it.
3. Read the broken page and find the errors: the h1 tag is never closed, and the first paragraph ends with  instead of .
4. Correct both errors and save the page as attendance-fixed.html.
5. Open all files in the browser and check the output.

## 6. Program / Queries / Commands:

**Task 1:** layout.html - header, main and footer blocks.

**layout.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 4 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Layout</title>
  <style>
    div { border: 1px solid #555; padding: 10px; margin: 6px 0; }
  </style>
</head>
<body>
  <div class="header">This is the header.</div>
  <div class="main">This is the main content.</div>
  <div class="footer">This is the footer.</div>
</body>
</html>
```

**Task 2:** attributes.html - id and a data attribute.

**attributes.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 4 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Attributes</title>
</head>
<body>
  <div id="profile">
    <p data-topic="hobbies">My hobbies are coding, cricket and listening to music.</p>
  </div>
</body>
</html>
```

**Task 3:** attendance-fixed.html - the corrected page (errors fixed: </h1> added,  changed to ).

**attendance-fixed.html**

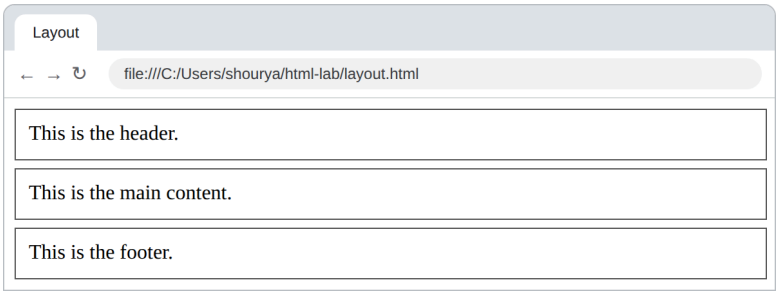
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 4 | Task: 3 -->
<!DOCTYPE html>
<html>
<head>
<title>Attendance</title>
</head>
<body>
<h1>Attendance Report</h1>
<p>Asha: 92%</p>
<p>Rohan: 85%</p>
</body>
</html>
```

## 7. Input:

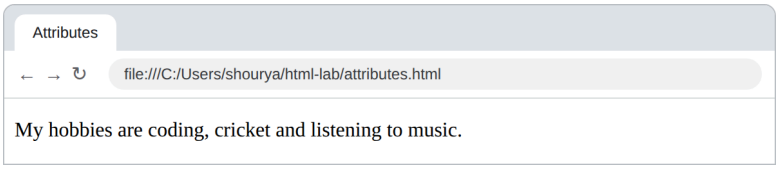
No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

**Task 1:** Three blocks in order: header, main and footer.




**Task 2:** The paragraph text shows. The data-topic attribute is not visible on the page.




**Task 3:** The corrected page shows the heading Attendance Report and two lines: Asha: 92% and Rohan: 85%.




## 9. Result:

The div blocks and the data attribute worked as expected, and both errors in the broken page were found and corrected. The experiment was performed successfully.

# EXPERIMENT NO. 5

### 1. Experiment Title:

Links and Images

### 2. Aim:

To link two pages together and to add a picture with alt text.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

The anchor element **a** creates a hyperlink; its **href** attribute holds the address of the target. A file in the same folder is linked by its file name (relative link). The **img** element shows a picture from its **src** path; **width** and **height** set its size and **alt** gives a text description that appears if the picture cannot load and is read by screen readers.

### 5. Algorithm / Procedure:

1. Create home.html with a heading and a paragraph containing a link (text: portfolio) to portfolio.html.
2. Create portfolio.html with a heading and a link back to home.html.
3. Place a picture in the images folder.
4. Create portfolio-image.html with a heading and an img tag with width="200", height="200" and alt text.
5. Open home.html, click the link, click back, and open portfolio-image.html.

### 6. Program / Queries / Commands:

**Task 1:** home.html - heading and a link to portfolio.html.

**home.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 5 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home</title>
</head>
<body>
  <h1>Welcome to My Site</h1>
  <p>Click here to see my <a href="portfolio.html">portfolio</a>.</p>
</body>
</html>
```

**Task 2:** portfolio.html - heading and a link back to home.html.

**portfolio.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 5 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio</title>
</head>
<body>
  <h1>My Portfolio</h1>
  <p><a href="home.html">Go back to the home page</a></p>
</body>
</html>
```

**Task 3:** portfolio-image.html - a 200 x 200 picture with alt text.

**portfolio-image.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 5 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio Image</title>
</head>
<body>
  <h1>My Favourite Place</h1>
  <img src="images/photo.jpg" width="200" height="200"
       alt="Green hills under an orange evening sky with the sun setting">
</body>
</html>
```

## 7. Input:

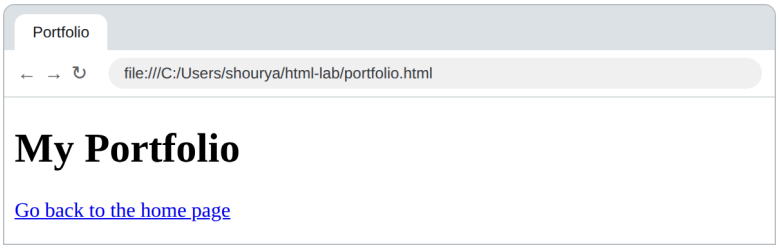
The picture file used is images/photo.jpg (use the file name given by the instructor).

## 8. Output:

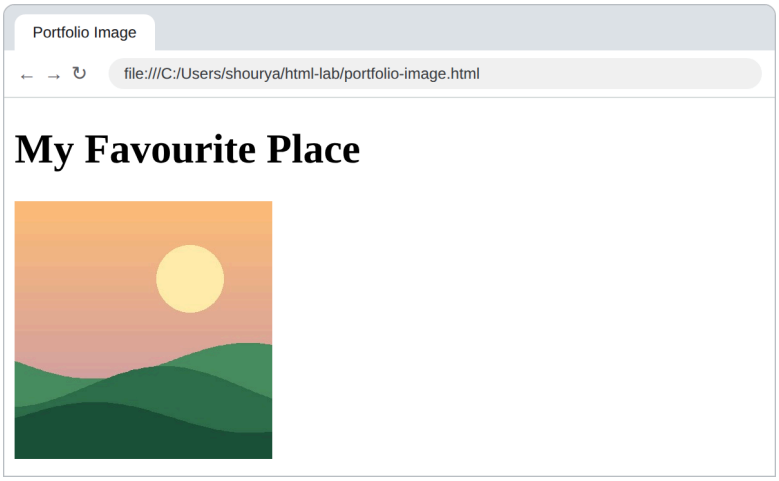
**Task 1:** Clicking the word portfolio opens the portfolio page.




**Task 2:** The link goes back to the home page.




**Task 3:** A 200 by 200 photo under the heading. If the picture is missing, the alt text shows instead.




## 9. Result:

The two pages were linked in both directions and the image was displayed at 200 x 200 pixels with alt text. The experiment was performed successfully.

# EXPERIMENT NO. 6

### 1. Experiment Title:

Types of HTML Lists

### 2. Aim:

To make a bulleted list, a numbered list with a nested list, and a description list.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

**ul** makes an unordered (bulleted) list and **ol** makes an ordered (numbered) list; every item is written in an **li** element. A list can be placed inside an li to make a nested list. A description list **dl** pairs a term **dt** with its description **dd**, which suits questions and answers.

### 5. Algorithm / Procedure:

1. Create shopping-list.html with an h1 and a ul of three items.
2. Create canteen-menu.html with an ol of two items and a nested ol under Sandwich.
3. Create faq.html with a dl containing two dt (question) and two dd (answer) elements.
4. Open the files in the browser and take screenshots.

### 6. Program / Queries / Commands:

**Task 1:** shopping-list.html - unordered list.

**shopping-list.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 6 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Shopping List</title>
</head>
<body>
  <h1>Shopping List</h1>
  <ul>
    <li>Notebook</li>
    <li>Pen</li>
    <li>Bag</li>
  </ul>
</body>
</html>
```

**Task 2:** canteen-menu.html - numbered list with a nested numbered list.

**canteen-menu.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 6 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Canteen Menu</title>
</head>
<body>
  <h1>Canteen Menu</h1>
  <ol>
    <li>Samosa</li>
    <li>Sandwich
      <ol>
        <li>Cheese</li>
        <li>Vegetable</li>
```

```
      </ol>
    </li>
  </ol>
</body>
</html>
```

**Task 3:** faq.html - description list of two questions and answers.

**faq.html**

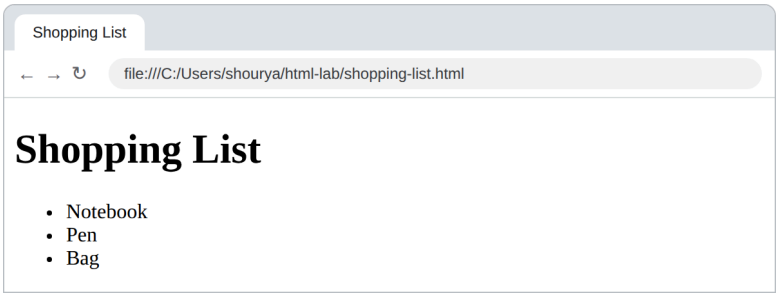
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 6 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Library FAQ</title>
</head>
<body>
  <h1>Library FAQ</h1>
  <dl>
    <dt>How many books can I borrow?</dt>
    <dd>You can borrow up to three books at a time.</dd>
    <dt>Is the library open on Sunday?</dt>
    <dd>No, the library is closed on Sunday.</dd>
  </dl>
</body>
</html>
```

## 7. Input:

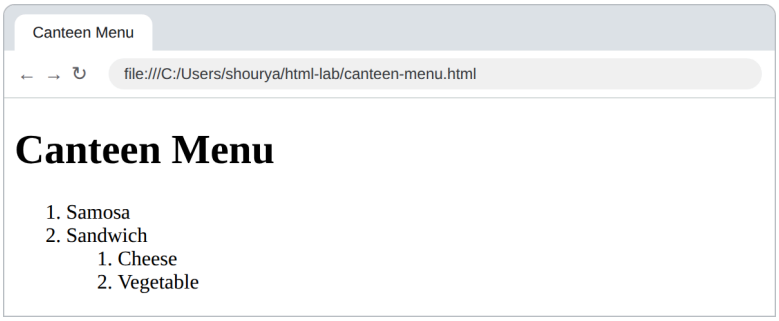
No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

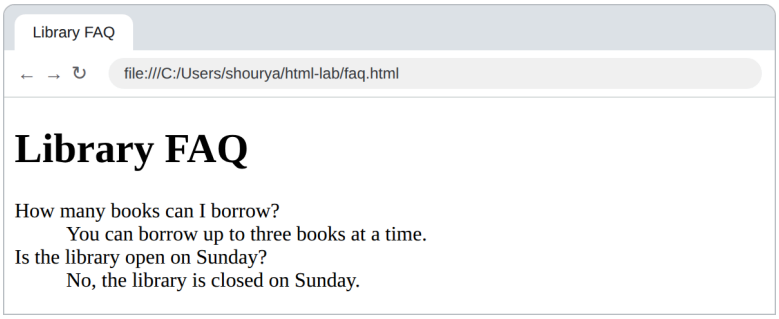
**Task 1:** Three bullet points.




**Task 2:** Two numbered items, and a smaller numbered list under the second one.



**Task 3:** Two questions, each followed by its answer.




## 9. Result:

The bulleted list, the numbered list with a nested list and the description list were displayed correctly. The experiment was performed successfully.

# EXPERIMENT NO. 7

### 1. Experiment Title:

About Me Page

### 2. Aim:

To build a short About Me page with semantic tags, an about section and a contact link.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

Semantic elements describe the meaning of each part of a page: **header** (top area), **nav** (navigation links), **main** (main content), **section** (a themed group of content) and **footer** (bottom area). A link such as **href="#about"** jumps to the element with that id on the same page, and **href="mailto:..."** opens the email program.

### 5. Algorithm / Procedure:

1. Create about-me.html with a header (h1 with the name and a nav with links to #about and #contact), an empty main and an empty footer.
2. Add a section with id about inside main, with an h2 "About me" and a paragraph.
3. Give the footer the id contact and add a paragraph with a mailto: link.
4. Open the page and click the Contact link to check that it jumps to the footer.

### 6. Program / Queries / Commands:

**Task 1:** about-me.html - header with nav, empty main and empty footer.

**about-me.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 7 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Abhijeet Kumar</    h1>
    <nav>
      <a href="#about">About</a> |
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
  </main>
  <footer>
  </footer>
</body>
</html>
```

**Task 2:** Add the about section inside main (about-me.html).

**about-me.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 7 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Abhijeet Kumar</    h1>
    <nav>
```

```
      <a href="#about">About</a> |
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h2>About me</h2>
      <p>I am a first-year B.Tech CSE student at Invertis University, Bareilly.
         I am learning web development and I like building small projects.</p>
    </section>
  </main>
  <footer>
  </footer>
</body>
</html>
```

**Task 3:** Add the contact paragraph with a mailto link in the footer (final about-me.html).

**about-me.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 7 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <header>
    <h1>Abhijeet Kumar</    h1>
    <nav>
      <a href="#about">About</a> |
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about">
      <h2>About me</h2>
      <p>I am a first-year B.Tech CSE student at Invertis University, Bareilly.
         I am learning web development and I like building small projects.</p>
    </section>
  </main>
  <footer id="contact">
    <p>Email me at <a href="mailto:abhijeet@example.com" >abhijeet@example.com</ a></p>
  </footer>
</body>
</html>
```

## 7. Input:

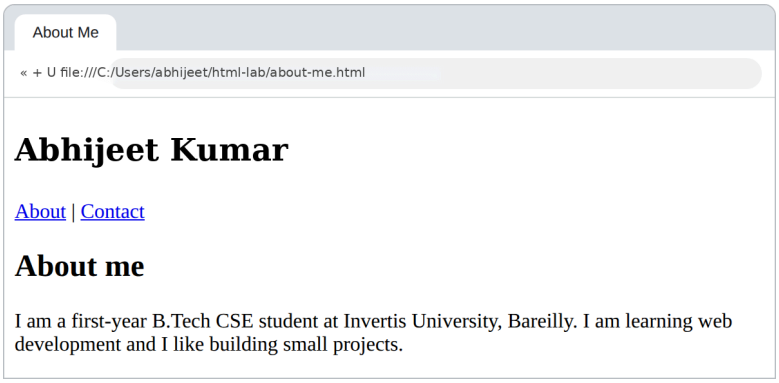
Email address used in the link: abhijeet@example.com

## 8. Output:

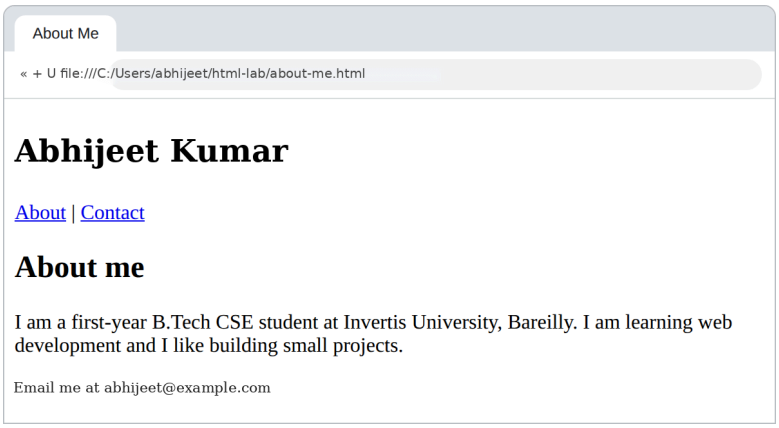
**Task 1:** Your name at the top with two links. The middle and bottom are empty.



**Task 2:** The About me heading and a paragraph appear under the top links.




**Task 3:** The footer shows a clickable email address. The Contact link in the top nav jumps to it.




## 9. Result:

The About Me page was built step by step with header, nav, main, section and footer. The Contact link jumps to the footer and the email link is clickable. The experiment was performed successfully.

# EXPERIMENT NO. 8

### 1. Experiment Title:

Sign-Up Forms and Choices

### 2. Aim:

To make a labelled form with text, email and password fields, choices and a submit button.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

A **form** collects user input and sends it to the address in its **action** using the **method** (post). A **label** is linked to a field with **for** = the field's **id**, so clicking the label selects the field. **input** types include text, email and password (hides the typed characters). Radio buttons with the same **name** allow only one choice; **select** with **option** makes a dropdown. **required** stops the form from being submitted when a field is empty and **placeholder** shows a hint inside the field.

### 5. Algorithm / Procedure:

1. Create signup.html with a form (method post, action thanks.html) and labelled full name, email and password fields.
2. Create choices.html with two radio buttons (Female, Male) with the same name, and a select with BCA and B.Tech.
3. Create signup-required.html with a required full name field (with placeholder), a required email field and a Submit button.
4. Open each file; test the form by typing values and by submitting with empty fields.

### 6. Program / Queries / Commands:

**Task 1:** signup.html - form with three labelled fields.

**signup.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 8 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sign Up</title>
</head>
<body>
  <h1>Sign Up</h1>
  <form method="post" action="thanks.html">
    <p>
      <label for="fullname">Full name:</label><br>
      <input type="text" id="fullname" name="fullname">
    </p>
    <p>
      <label for="email">Email:</label><br>
      <input type="email" id="email" name="email">
    </p>
    <p>
      <label for="password">Password:</label><br>
      <input type="password" id="password" name="password">
    </p>
  </form>
</body>
</html>
```

**Task 2:** choices.html - radio buttons and a dropdown.

**choices.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 8 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Choices</title>
</head>
<body>
  <h1>Choices</h1>
  <form>
    <p>Gender:<br>
      <input type="radio" id="female" name="gender" value="female">
      <label for="female">Female</label>
      <input type="radio" id="male" name="gender" value="male">
      <label for="male">Male</label>
    </p>
    <p>
      <label for="course">Course:</label><br>
      <select id="course" name="course">
        <option value="bca">BCA</option>
        <option value="btech">B.Tech</option>
      </select>
    </p>
  </form>
</body>
</html>
```

**Task 3:** signup-required.html - required fields, placeholder and Submit button.

**signup-required.html**

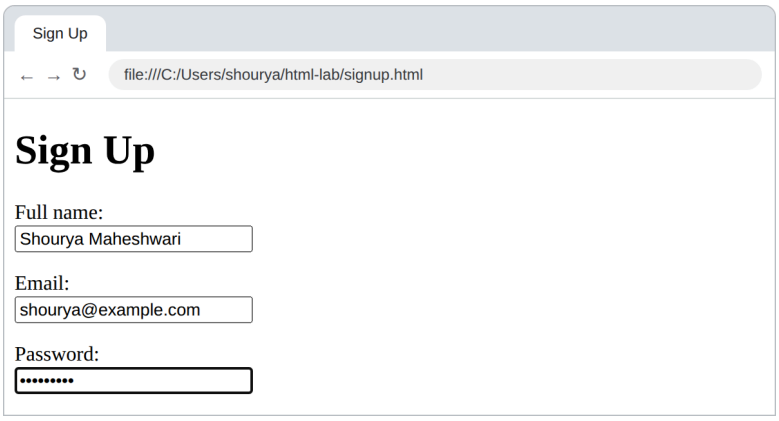
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 8 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sign Up (Required)</title>
</head>
<body>
  <h1>Join the Club</h1>
  <form>
    <p>
      <label for="fullname">Full name:</label><br>
      <input type="text" id="fullname" name="fullname"
             placeholder="Enter your full name" required>
    </p>
    <p>
      <label for="email">Email:</label><br>
      <input type="email" id="email" name="email" required>
    </p>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

## 7. Input:

Test data typed in Task 1: Full name: Abhijeet Kumar, Email: abhijeet@example.com, Password: secret123

## 8. Output:

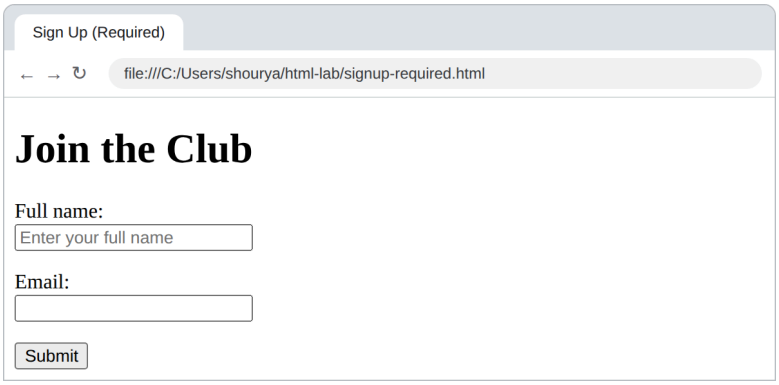
**Task 1:** Three labelled boxes. The password box hides the characters you type.




**Task 2:** Only one gender can be selected. The dropdown shows the two courses.




**Task 3:** Leaving a required field empty shows a message from the browser ("Please fill out this field").




## 9. Result:

The sign-up form, the radio buttons, the dropdown and the required-field form worked correctly. The experiment was performed successfully.

# EXPERIMENT NO. 9

## 1. Experiment Title:

Tables with Headers and Footers

## 2. Aim:

To build a table with a caption, a header, rows and a footer, and to add scope attributes.

## 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

## 4. Theory:

A **table** is made of rows (**tr**) that hold header cells (**th**) and data cells (**td**). **caption** gives the table a title. **thead**, **tbody** and **tfoot** separate the header, the body and the footer. The **scope** attribute (col or row) tells screen readers whether a header cell describes a column or a row.

## 5. Algorithm / Procedure:

1. Create marks.html with a caption, a thead (Name, Maths, Science) and a tbody with three students.
2. Create marks-average.html with Asha and Rohan only and a tfoot row "Class average". Maths average = (78 + 64) / 2 = 71; Science average = (85 + 70) / 2 = 77.5.
3. Create marks-accessible.html with the same table without the footer, adding scope="col" to the header cells and scope="row" to the name cells.
4. Open the files in the browser and take screenshots.

## 6. Program / Queries / Commands:

**Task 1:** marks.html - table with caption, thead and tbody.

**marks.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 9 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Marks</title>
  <style>
    table { border-collapse: collapse; }
    th, td { border: 1px solid #444; padding: 6px 14px; text-align: left; }
    thead th, tfoot th, tfoot td { background: #e8eef7; }
  </style>
</head>
<body>
  <table>
    <caption>Marks of Class A</caption>
    <thead>
      <tr><th>Name</th><th>Maths</th><th>Science</th></tr>
    </thead>
    <tbody>
      <tr><td>Asha</td><td>78</td><td>85</td></tr>
      <tr><td>Rohan</td><td>64</td><td>70</td></tr>
      <tr><td>Neha</td><td>91</td><td>88</td></tr>
    </tbody>
  </table>
</body>
</html>
```

**Task 2:** marks-average.html - tfoot row with the class average.

**marks-average.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 9 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Marks with Average</title>
  <style>
    table { border-collapse: collapse; }
    th, td { border: 1px solid #444; padding: 6px 14px; text-align: left; }
    thead th, tfoot th, tfoot td { background: #e8eef7; }
  </style>
</head>
<body>
  <table>
    <caption>Marks of Class A</caption>
    <thead>
      <tr><th>Name</th><th>Maths</th><th>Science</th></tr>
    </thead>
    <tbody>
      <tr><td>Asha</td><td>78</td><td>85</td></tr>
      <tr><td>Rohan</td><td>64</td><td>70</td></tr>
    </tbody>
    <tfoot>
      <tr><td>Class average</td><td>71</td><td>77.5</td></tr>
    </tfoot>
  </table>
</body>
</html>
```

**Task 3:** marks-accessible.html - scope attributes on header and name cells.

**marks-accessible.html**

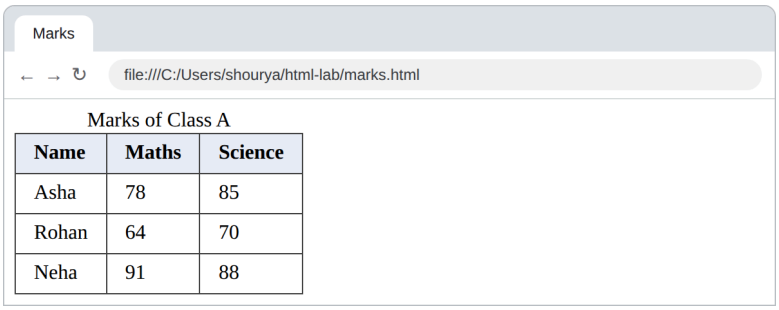
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 9 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Accessible Marks</title>
  <style>
    table { border-collapse: collapse; }
    th, td { border: 1px solid #444; padding: 6px 14px; text-align: left; }
    thead th, tfoot th, tfoot td { background: #e8eef7; }
  </style>
</head>
<body>
  <table>
    <caption>Marks of Class A</caption>
    <thead>
      <tr>
        <th scope="col">Name</th>
        <th scope="col">Maths</th>
        <th scope="col">Science</th>
      </tr>
    </thead>
    <tbody>
      <tr><th scope="row">Asha</th><td>78</td><td>85</td></tr>
      <tr><th scope="row">Rohan</th><td>64</td><td>70</td></tr>
    </tbody>
  </table>
</body>
</html>
```

## 7. Input:

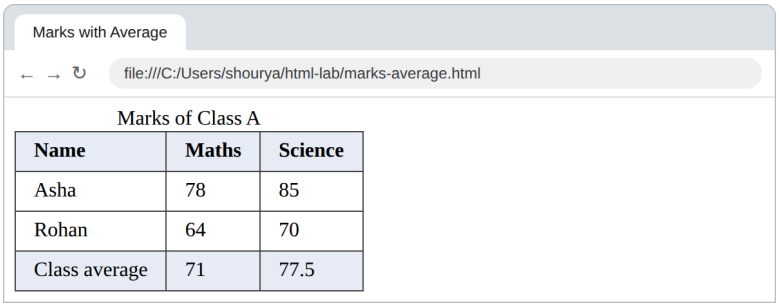
Marks: Asha 78, 85; Rohan 64, 70; Neha 91, 88.

## 8. Output:

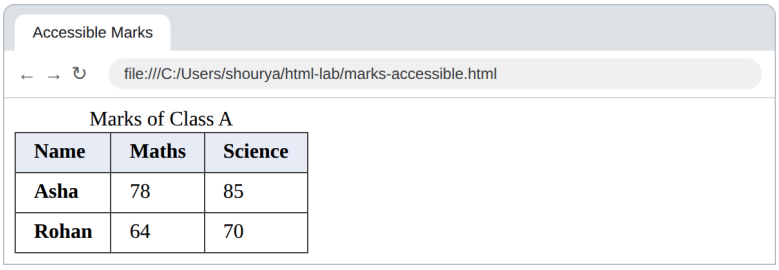
**Task 1:** A table with a header row and three student rows.




**Task 2:** The footer row at the bottom shows the two averages (71 and 77.5).




**Task 3:** The table looks the same. The header and name cells carry scope attributes (visible in the Elements panel).




## 9. Result:

The tables were created with caption, header, body, footer and scope attributes, and the class averages (71 and 77.5) were shown in the footer. The experiment was performed successfully.

# EXPERIMENT NO. 10

### 1. Experiment Title:

CSS Fundamentals

### 2. Aim:

To apply CSS in three ways (inline, internal and linked) and to use selectors and text styles.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

CSS controls how HTML looks. It can be written *inline* (the style attribute on one element), *internal* (a style block in the head) or *external* (a .css file connected with **<link rel="stylesheet">**). A **type selector** (h1) styles every element of that type, a **class selector** (.note) styles elements with that class and an **id selector** (#title) styles the one element with that id. Properties used: color, background-color, font-style, text-decoration and font-size.

### 5. Algorithm / Procedure:

1. Create inline-internal.html with an internal style block (h1 navy, .highlight yellow background), an h1, a paragraph with an inline red style and a paragraph with the class highlight.
2. Create style.css with rules for h1, .note, #title and p.
3. Create linked.html that links style.css, with an h1 (id title) and two paragraphs (class note).
4. Keep style.css in the same folder as linked.html and open the page in the browser.

### 6. Program / Queries / Commands:

**Task 1:** inline-internal.html - inline and internal CSS.

**inline-internal.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 10 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Inline and Internal CSS</title>
  <style>
    h1 { color: navy; }
    .highlight { background-color: yellow; }
  </style>
</head>
<body>
  <h1>CSS in Two Ways</h1>
  <p style="color: red;">This paragraph is red because of an inline style.</p>
  <p class="highlight">This paragraph has a yellow background from the internal style.</p>
</body>
</html>
```

**Task 2:** style.css - external style sheet.

**style.css**

```
/* Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 10 | Task: 2 */
h1 {
  color: navy;
}

.note {
  font-style: italic;
}

#title {
  text-decoration: underline;
}
```

```css
p {
  font-size: 18px;
  color: grey;
}
```

**Task 3:** linked.html - page that uses style.css.

**linked.html**

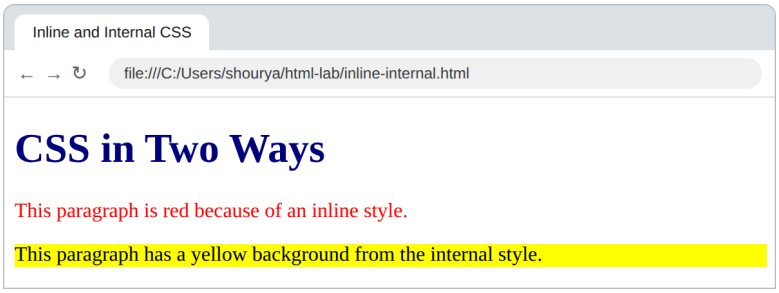
```html
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 10 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Linked CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="title">Linked Style Sheet</h1>
  <p class="note">This note is italic and grey.</p>
  <p class="note">This is another note styled by style.css.</p>
</body>
</html>
```

## 7. Input:

No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

**Task 1:** A navy heading, one red paragraph and one yellow paragraph.




**Task 2:** style.css has no output of its own. Once linked (Task 3), the headings, notes and paragraphs use the new styles.

**Task 3:** The page is styled only when style.css is in the same folder.




## 9. Result:

Inline, internal and external CSS were all applied successfully, and the type, class and id selectors styled the elements as required. The experiment was performed successfully.

# EXPERIMENT NO. 11

### 1. Experiment Title:

Units and CSS Variables

### 2. Aim:

To use CSS units and to store colours and spacing in variables.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

**px** is a fixed unit, **%** is relative to the parent element's width and **rem** is relative to the root font size (1rem is normally 16px). CSS variables (custom properties) are defined with a name starting with **--**, usually inside **:root**, and used with **var(--name)**. Changing the variable in one place changes every use of it.

### 5. Algorithm / Procedure:

1. Create units.html with a 200px wide box, a 50% wide box and a paragraph at 1.5rem.
2. Create variables.html and define --main-color, --accent-color and --page-padding in :root; use var() for the body colour and padding and the h1 colour.
3. Create tokens.css with the primary colour, accent colour, heading size and spacing variables and use them for body and h1.
4. Link tokens.css to a test page (tokens-demo.html) and open all pages.

### 6. Program / Queries / Commands:

**Task 1:** units.html - px, % and rem.

**units.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 11 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Units</title>
  <style>
    .box-px  { width: 200px; height: 40px; background-color: #f4a261; }
    .box-pct { width: 50%;   height: 40px; background-color: #2a9d8f; margin-top: 10px; }
    p { font-size: 1.5rem; }
  </style>
</head>
<body>
  <div class="box-px"></div>
  <div class="box-pct"></div>
  <p>This text is 1.5rem in size.</p>
</body>
</html>
```

**Task 2:** variables.html - variables defined in :root.

**variables.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 11 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Variables</title>
  <style>
    :root {
      --main-color: #1d3557;
      --accent-color: #e63946;
      --page-padding: 20px;
    }
```

```
    body {
      color: var(--main-color);
      padding: var(--page-padding);
    }
    h1 {
      color: var(--accent-color);
    }
  </style>
</head>
<body>
  <h1>Using CSS Variables</h1>
  <p>This paragraph uses the main colour stored in a variable.</p>
</body>
</html>
```

**Task 3:** tokens.css (with tokens-demo.html to show it).

**tokens.css**

```
/* Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 11 | Task: 3 */
:root {
  --primary-color: #1d3557;
  --accent-color: #e63946;
  --heading-size: 2rem;
  --spacing: 16px;
}

body {
  color: var(--primary-color);
  padding: var(--spacing);
}

h1 {
  color: var(--accent-color);
  font-size: var(--heading-size);
}
```

**tokens-demo.html**

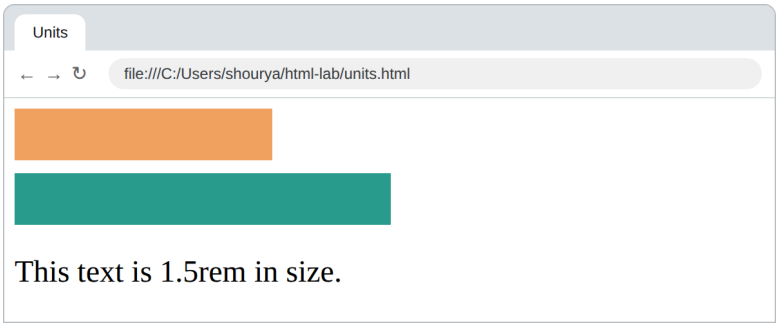
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 11 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tokens Demo</title>
  <link rel="stylesheet" href="tokens.css">
</head>
<body>
  <h1>Design Tokens</h1>
  <p>The colours, heading size and spacing come from tokens.css.</p>
</body>
</html>
```

## 7. Input:

No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

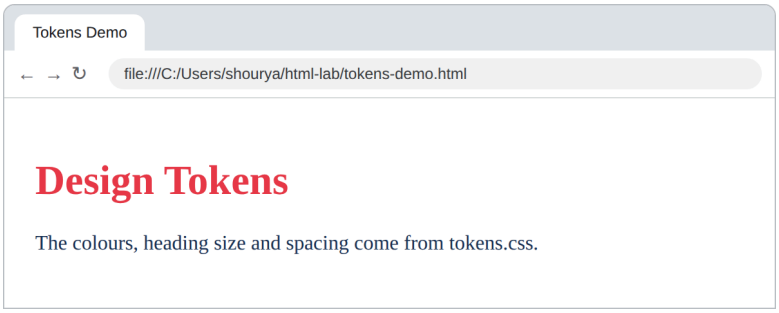
**Task 1:** A 200 pixel box, a half-width box and larger text.



**Task 2:** The heading uses the accent colour. The text uses the main colour.




**Task 3:** Linked to a page, the file sets the colours, heading size and spacing.




## 9. Result:

CSS units (px, %, rem) and CSS variables were used successfully, both inside a page and in a separate tokens.css file. The experiment was performed successfully.

# EXPERIMENT NO. 12

### 1. Experiment Title:

Box Model, Cards and Scrolling

### 2. Aim:

To compare the box sizing types, to style a card and to make a box scroll.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

Every element is a box made of *content*, *padding*, *border* and *margin*. With **box-sizing: content-box** the width covers only the content, so the total width is width + padding + border (200 + 40 + 10 = 250px). With **border-box** the width includes padding and border, so the total stays 200px. **border-radius** rounds the corners and **overflow: auto** adds a scroll bar when the content is larger than the box.

### 5. Algorithm / Procedure:

1. Create box-model.html with two boxes (width 200px, padding 20px, border 5px); give the first content-box and the second border-box.
2. Create card.html with a card (background colour, 2px border, border-radius 12px, padding 16px) holding an h3 and a paragraph.
3. Create scroll.html with a box of height 120px and overflow auto holding five paragraphs, Line 1 to Line 5.
4. Open the files in the browser and compare the output.

### 6. Program / Queries / Commands:

**Task 1:** box-model.html - content-box and border-box.

**box-model.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 12 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Box Model</title>
  <style>
    .box {
      width: 200px;
      padding: 20px;
      border: 5px solid #1d3557;
      background-color: #a8dadc;
      margin-bottom: 12px;
    }
    .first  { box-sizing: content-box; }
    .second { box-sizing: border-box; }
  </style>
</head>
<body>
  <div class="box first">content-box</div>
  <div class="box second">border-box</div>
</body>
</html>
```

**Task 2:** card.html - a styled card.

**card.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 12 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Card</title>
```

```
  <style>
    .card {
      background-color: #f1faee;
      border: 2px solid #457b9d;
      border-radius: 12px;
      padding: 16px;
      width: 260px;
    }
  </style>
</head>
<body>
  <div class="card">
    <h3>Web Development</h3>
    <p>Learn HTML and CSS to build your own web pages.</p>
  </div>
</body>
</html>
```

**Task 3:** scroll.html - a scrollable box.

**scroll.html**

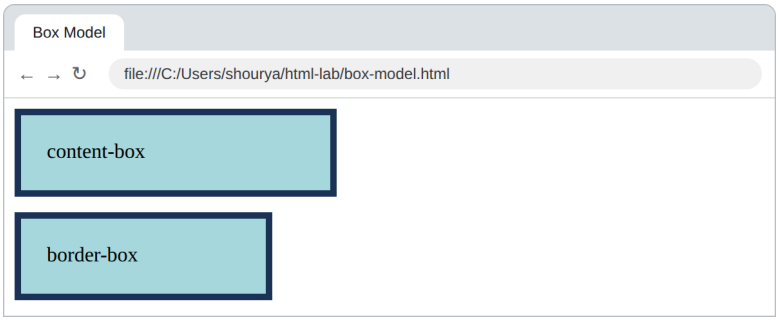
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 12 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Scroll Box</title>
  <style>
    .scroll-box {
      height: 120px;
      width: 240px;
      overflow: auto;
      border: 2px solid #333;
      padding: 0 10px;
    }
  </style>
</head>
<body>
  <div class="scroll-box">
    <p>Line 1</p>
    <p>Line 2</p>
    <p>Line 3</p>
    <p>Line 4</p>
    <p>Line 5</p>
  </div>
</body>
</html>
```

## 7. Input:

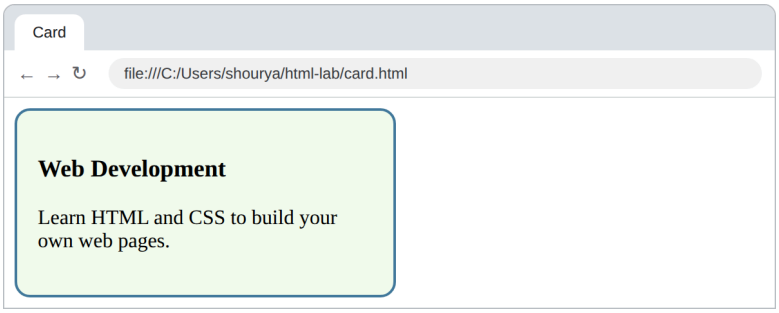
No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

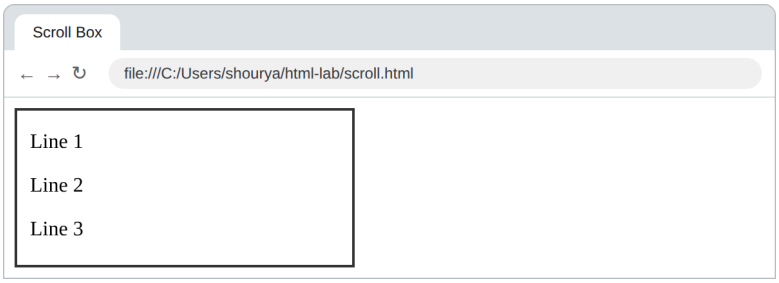
**Task 1:** The content-box box (250px in total) is wider than the border-box box (200px in total).



**Task 2:** A card with rounded corners, a border and a coloured background.




**Task 3:** The lines do not fit in the box, so the box can be scrolled.




## 9. Result:

The difference between content-box and border-box was observed, a styled card was created and a scrollable box was made with overflow. The experiment was performed successfully.

# EXPERIMENT NO. 13

### 1. Experiment Title:

Flexbox Layouts

### 2. Aim:

To lay out a navigation bar, centred cards and cards that wrap, using Flexbox.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

Flexbox arranges the children of a container in a row or column. **display: flex** turns an element into a flex container. **justify-content** aligns the items along the main axis and **align-items** aligns them along the cross axis (both set to *center* here). **flex-wrap: wrap** lets items move to a new row when there is not enough space, and **gap** sets the space between items.

### 5. Algorithm / Procedure:

1. Create nav-flex.html with a nav (display flex, dark background) holding four white links.
2. Create centred-cards.html with a .row (flex, justify-content and align-items centre) holding three cards.
3. Create wrapping-cards.html with the heading Stationery Shop and four cards in a flex-wrap row.
4. Open the files and narrow the browser window to check the wrapping.

### 6. Program / Queries / Commands:

**Task 1:** nav-flex.html - flex navigation bar.

**nav-flex.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 13 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Flex Navigation</title>
  <style>
    body { margin: 0; font-family: Arial, sans-serif; }
    nav {
      display: flex;
      gap: 24px;
      background-color: #222;
      padding: 14px 20px;
    }
    nav a { color: white; text-decoration: none; }
  </style>
</head>
<body>
  <nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Events</a>
    <a href="#">Contact</a>
  </nav>
</body>
</html>
```

**Task 2:** centred-cards.html - three cards centred in a row.

**centred-cards.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 13 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Centred Cards</title>
```

```
  <style>
    .row {
      display: flex;
      justify-content: center;
      align-items: center;
      gap: 16px;
      height: 240px;
      background-color: #eef2f7;
    }
    .card {
      width: 130px;
      padding: 12px;
      text-align: center;
      background-color: white;
      border: 2px solid #457b9d;
      border-radius: 10px;
    }
  </style>
</head>
<body>
  <div class="row">
    <div class="card"><h3>Starter</h3><p>Rs. 199</p></div>
    <div class="card"><h3>Standard</h3><p>Rs. 349</p></div>
    <div class="card"><h3>Premium</h3><p>Rs. 499</p></div>
  </div>
</body>
</html>
```

**Task 3:** wrapping-cards.html - cards that wrap on a narrow window.

**wrapping-cards.html**

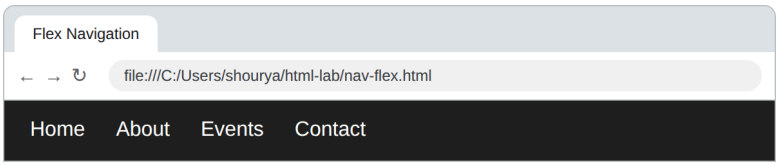
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 13 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Stationery Shop</title>
  <style>
    .cards {
      display: flex;
      flex-wrap: wrap;
      gap: 14px;
    }
    .card {
      width: 120px;
      padding: 12px;
      text-align: center;
      background-color: #f1faee;
      border: 2px solid #457b9d;
      border-radius: 10px;
    }
  </style>
</head>
<body>
  <h1>Stationery Shop</h1>
  <div class="cards">
    <div class="card"><h3>Notebook</h3><p>Rs. 60</p></div>
    <div class="card"><h3>Pen</h3><p>Rs. 15</p></div>
    <div class="card"><h3>Bag</h3><p>Rs. 350</p></div>
    <div class="card"><h3>Book</h3><p>Rs. 120</p></div>
  </div>
</body>
</html>
```

## 7. Input:

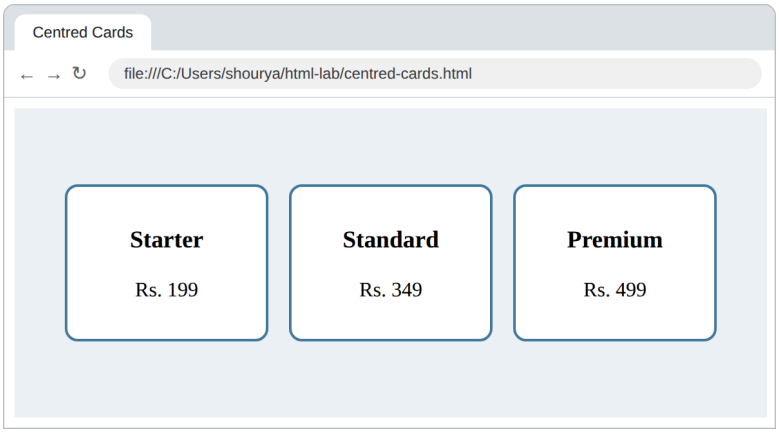
Prices used: Starter 199, Standard 349, Premium 499; Notebook 60, Pen 15, Bag 350, Book 120.

## 8. Output:

**Task 1:** A dark bar with four white links side by side.



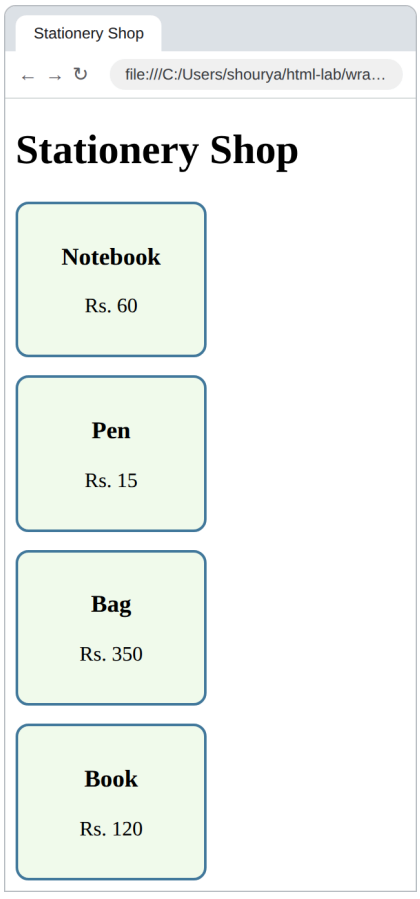

**Task 2:** Three cards in one row, centred in the row.




**Task 3:** On a wide window the four cards sit in one row; on a narrow window they wrap onto new rows.




*Wide window*




*Narrow window (cards wrap)*

## 9. Result:

The flex navigation bar, the centred cards and the wrapping cards worked as expected. The experiment was performed successfully.

# EXPERIMENT NO. 14

### 1. Experiment Title:

CSS Grid Layouts

### 2. Aim:

To make a grid with equal columns, a responsive gallery and a layout with named areas.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

CSS Grid arranges items in rows and columns. **grid-template-columns** defines the columns; **fr** units share the free space equally and **repeat()** avoids repeating values. **repeat(auto-fill, minmax(150px, 1fr))** fits as many columns of at least 150px as the window allows, which makes a responsive gallery. **grid-template-areas** names the areas of a layout and **grid-area** places an element in a named area.

### 5. Algorithm / Procedure:

1. Create grid-first.html with six boxes in a grid of three equal columns (1fr) and a 12px gap.
2. Create gallery.html with a heading and six boxes in a grid using repeat(auto-fill, minmax(150px, 1fr)) and a 10px gap.
3. Create dashboard-areas.html with columns 200px 1fr and grid-template-areas for header, sidebar, main and footer.
4. Open the files and resize the window for the gallery.

### 6. Program / Queries / Commands:

**Task 1:** grid-first.html - three equal columns.

**grid-first.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 14 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>First Grid</title>
  <style>
    .grid {
      display: grid;
      grid-template-columns: 1fr 1fr 1fr;
      gap: 12px;
    }
    .grid div {
      background-color: #a8dadc;
      border: 2px solid #1d3557;
      padding: 20px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="grid">
    <div>1</div><div>2</div><div>3</div>
    <div>4</div><div>5</div><div>6</div>
  </div>
</body>
</html>
```

**Task 2:** gallery.html - responsive auto-fill gallery.

**gallery.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 14 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Club Photo Gallery</title>
  <style>
    .gallery {
      display: grid;
      grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
      gap: 10px;
    }
    .gallery div {
      background-color: #e9c46a;
      padding: 40px 10px;
      text-align: center;
      border-radius: 8px;
    }
  </style>
</head>
<body>
  <h1>Club Photo Gallery</h1>
  <div class="gallery">
    <div>Photo 1</div><div>Photo 2</div><div>Photo 3</div>
    <div>Photo 4</div><div>Photo 5</div><div>Photo 6</div>
  </div>
</body>
</html>
```

**Task 3:** dashboard-areas.html - named grid areas.

**dashboard-areas.html**

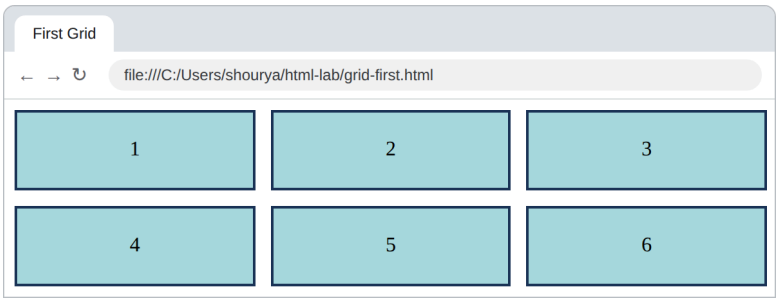
```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 14 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dashboard Areas</title>
  <style>
    .dashboard {
      display: grid;
      grid-template-columns: 200px 1fr;
      grid-template-rows: auto 1fr auto;
      grid-template-areas:
        "header  header"
        "sidebar main"
        "footer  footer";
      gap: 8px;
      min-height: 300px;
    }
    .header  { grid-area: header;  background-color: #457b9d; color: white; }
    .sidebar { grid-area: sidebar; background-color: #a8dadc; }
    .main    { grid-area: main;    background-color: #f1faee; }
    .footer  { grid-area: footer;  background-color: #1d3557; color: white; }
    .dashboard div { padding: 14px; }
  </style>
</head>
<body>
  <div class="dashboard">
    <div class="header">Header</div>
    <div class="sidebar">Sidebar</div>
    <div class="main">Main area</div>
    <div class="footer">Footer</div>
  </div>
</body>
</html>
```

## 7. Input:

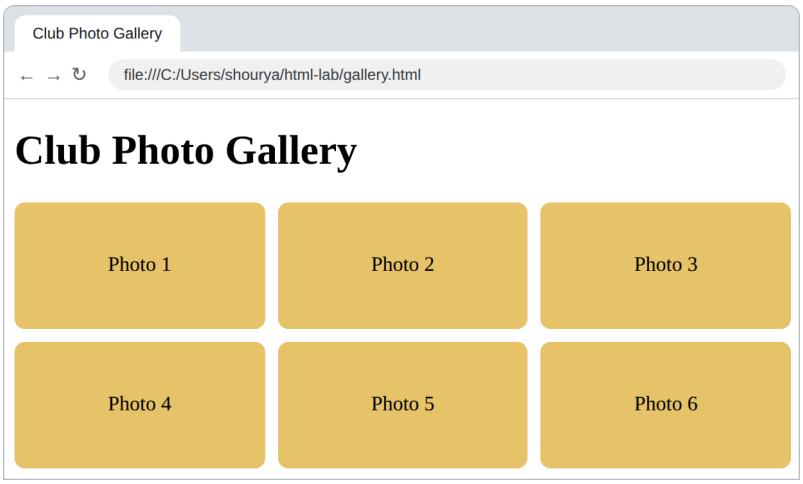
No keyboard input is needed. The file is opened directly in the browser.

## 8. Output:

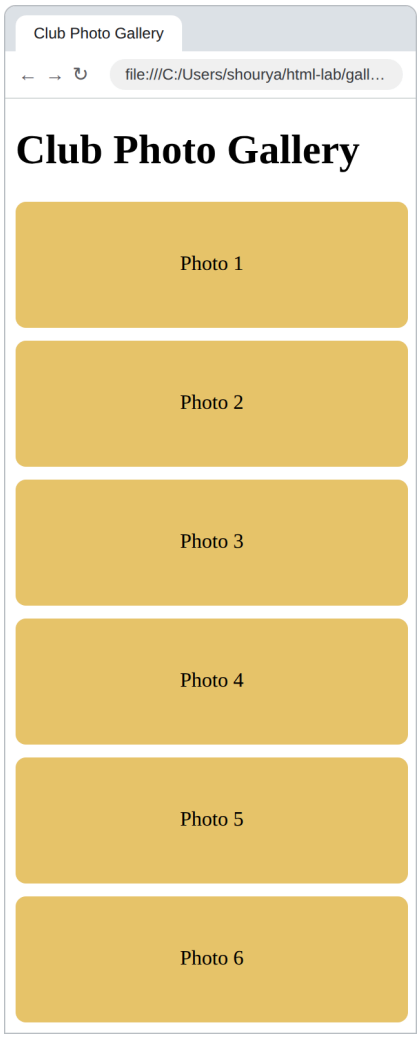
**Task 1:** Six boxes in two rows of three, with small gaps.




**Task 2:** As the window gets wider, more boxes fit in each row.

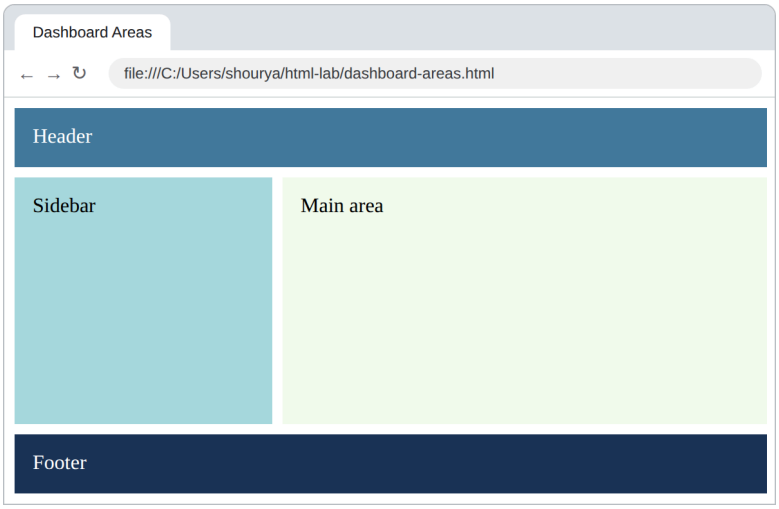



*Wide window*




*Narrow window*

**Task 3:** A header across the top, a sidebar on the left, a main area on the right and a footer at the bottom.




## 9. Result:

The equal-column grid, the responsive gallery and the named-area layout were created and displayed correctly. The experiment was performed successfully.

# EXPERIMENT NO. 15

### 1. Experiment Title:

Responsive Pages with Media Queries

### 2. Aim:

To use a media query to change a layout, to make a picture scale with its box, and to set text sizes with rem units.

### 3. Software / Tools Required:

HTML5 and CSS3; Visual Studio Code (code editor); Google Chrome (web browser)

### 4. Theory:

A responsive page adapts to the screen size. The **viewport meta tag** makes a phone use its real screen width. In the *mobile-first* approach the base styles are written for phones and a **media query** such as **@media (min-width: 600px)** adds the changes for wider screens. **max-width: 100%** with **height: auto** makes a picture shrink with its container, and **rem** units make text sizes depend on the root font size.

### 5. Algorithm / Procedure:

1. Create mobile-first.html with the viewport meta tag and two cards; write phone styles first, then a media query for min-width 600px that places the cards side by side.
2. Create responsive-image.html with an image set to max-width 100% and height auto.
3. Create responsive-text.html with body 1rem, h1 1.5rem and a media query for min-width 600px that sets h1 to 2.5rem.
4. Open each page in a wide and in a narrow window and compare.

### 6. Program / Queries / Commands:

**Task 1:** mobile-first.html - cards stack on phones, sit side by side from 600px.

**mobile-first.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 15 | Task: 1 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mobile First</title>
  <style>
    /* base styles for phones */
    .card {
      background-color: #f1faee;
      border: 2px solid #457b9d;
      border-radius: 10px;
      padding: 12px;
      margin-bottom: 12px;
    }

    /* screens 600px wide and up */
    @media (min-width: 600px) {
      .cards { display: flex; gap: 12px; }
      .card  { flex: 1; }
    }
  </style>
</head>
<body>
  <div class="cards">
    <div class="card"><h2>Card One</h2><p>This is the first card.</p></div>
    <div class="card"><h2>Card Two</h2><p>This is the second card.</p></div>
  </div>
</body>
</html>
```

**Task 2:** responsive-image.html - picture that scales with its box.

**responsive-image.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 15 | Task: 2 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Image</title>
  <style>
    img {
      max-width: 100%;
      height: auto;
    }
  </style>
</head>
<body>
  <h1>Evening Hills</h1>
  <img src="images/landscape.jpg" alt="Green hills under an orange evening sky">
</body>
</html>
```

**Task 3:** responsive-text.html - rem units and a media query.

**responsive-text.html**

```
<!-- Name: Abhijeet Kumar | Roll No: BCS2026 512 | Assignment: 15 | Task: 3 -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Text</title>
  <style>
    body { font-size: 1rem; }
    h1   { font-size: 1.5rem; }

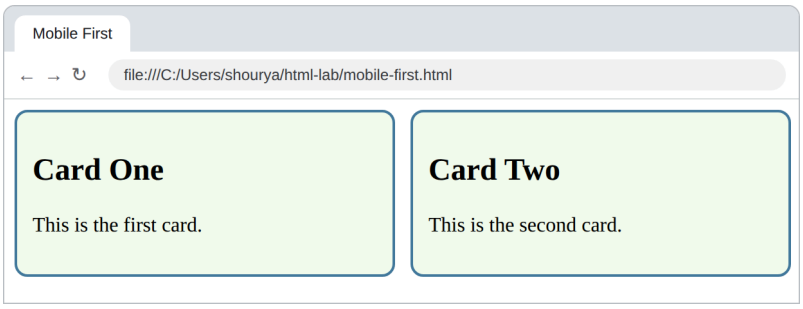
    @media (min-width: 600px) {
      h1 { font-size: 2.5rem; }
    }
  </style>
</head>
<body>
  <h1>Responsive Heading</h1>
  <p>This heading becomes bigger on wider screens.</p>
</body>
</html>
```

## 7. Input:

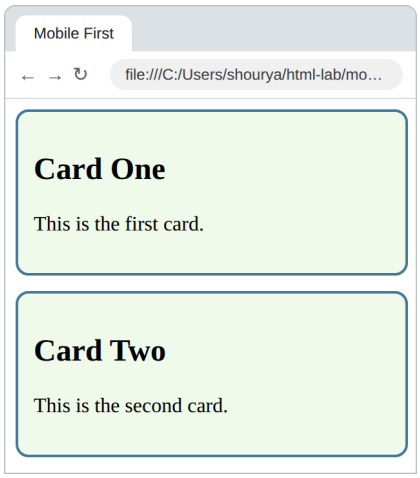
The picture file used is images/landscape.jpg (use the file name given by the instructor).

## 8. Output:

**Task 1:** On a phone the cards stack. At 600px and up they sit two per row.

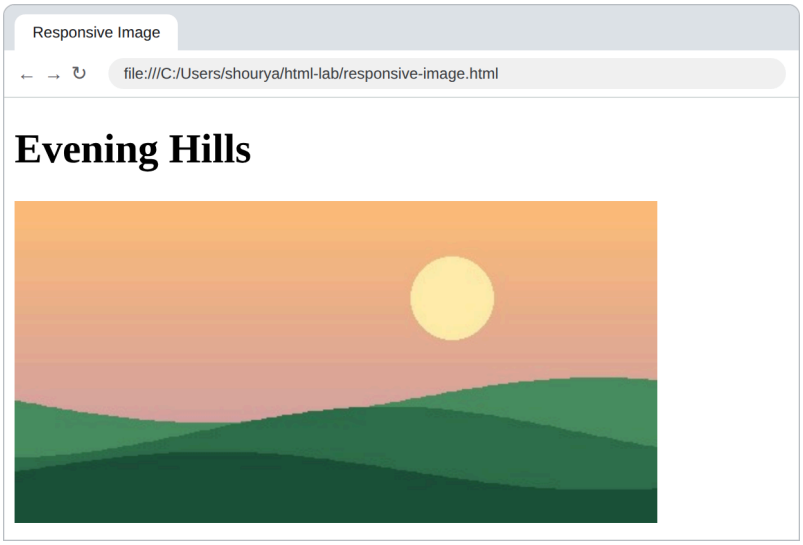



*Width 600px and up*

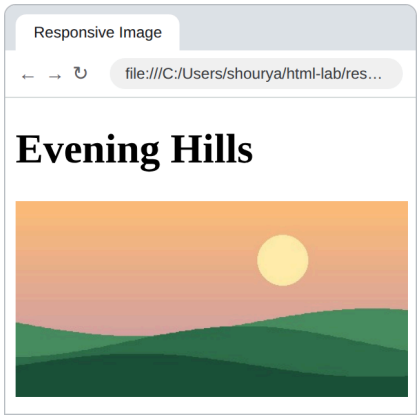



*Phone width*

**Task 2:** On a narrow window the picture shrinks to fit. On a wide window it keeps its own size.




*Wide window*

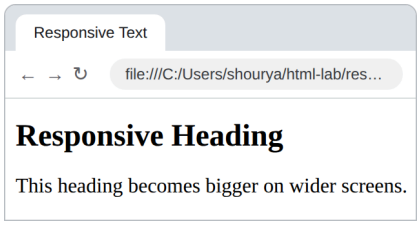



*Narrow window*

**Task 3:** The heading is larger when the window is 600px wide or more.




*Width 600px and up*




*Narrow window*

## 9. Result:

The media query changed the layout and the heading size at 600px, and the image scaled with its container. The experiment was performed successfully.